

数据库是可共享的资源，可以供多个用户使用。允许多个用户同时使用同一个数据库的数据库系统称为多用户数据库系统。例如飞机订票系统、银行系统、网上购物系统等都是多用户数据库系统。在这样的系统中，在同一时刻并发运行的事务数可达成千上万个。例如，淘宝应用在“双十一”购物节的并发量可以达到亿级。

事务可以一个一个地串行执行，即每个时刻只有一个事务运行，其他事务必须等到这个事务结束以后方能运行，如图 12.1 (a) 所示。事务在执行过程中需要不同的资源，有时需要 CPU，有时需要存取数据库，有时需要访问 I/O 设备，有时需要通信。如果事务串行执行，则许多系统资源将处于空闲状态。因此，为了充分利用系统资源，发挥数据库共享资源的特点，应该允许多个事务并行地执行。

在单处理机系统中，事务的并行执行实际上是这些并发事务的并行操作轮流交叉运行，如图 12.1 (b) 所示。这种并行执行方式称为交叉并发 (interleaved concurrency) 方式。虽然单处理机系统中的并行事务并没有真正地并行运行，但是减少了处理机的空闲时间，提高了系统的效率。

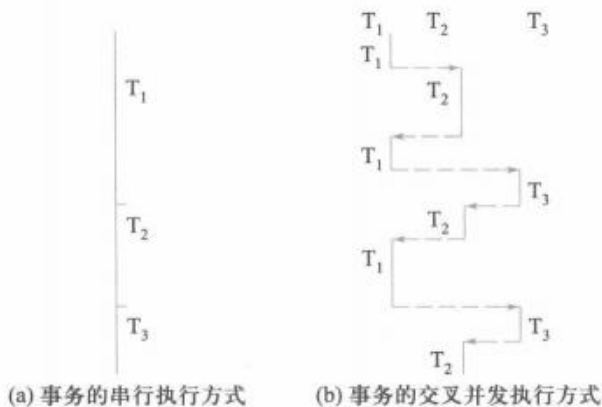


图 12.1 事务的执行方式

在多处理机系统中,每个处理机可以运行一个事务,多个处理机可以同时运行多个事务,实现多个事务真正的并行运行。这种并行执行方式称为同时并发(simultaneous concurrency)方式。本章讨论的数据库系统并发控制(concurrency control)技术是以单处理机系统为基础的,该理论可以推广到多处理机的情况。

当多个用户并发地存取数据库时,就会产生多个事务同时存取同一数据的情况。若对并发操作不加控制就可能会存取和存储不正确的数据,破坏事务的一致性和数据库的一致性。所以,数据库管理系统必须提供并发控制机制。并发控制机制是衡量一个数据库管理系统性能的重要标志之一。

本章导读

多用户共享是数据库的一个显著特点,而多用户同时访问同一数据对象可能会破坏事务的隔离性。并发控制机制就是要用正确的方式调度并发操作,使一个用户事务的执行不受其他事务的干扰,从而避免造成数据的不一致性。

本章讨论并发控制的概念和常用技术。首先介绍并发操作带来的数据不一致问题,这些问题都是由于事务的隔离性没有得到保障造成的,所以接下来介绍事务的4种隔离级别。并发控制的方法有多种,本章重点讲解封锁技术,包括封锁的概念、封锁协议及其对数据一致性的保证、活锁与死锁及其解决方案、多粒度封锁等。本章还讨论了并发调度的可串行性,以及如何用封锁技术保证并发事务的可串行性。

学习完本章内容,要了解并发操作造成的数据不一致性,理解并发调度的可串行性,深入掌握封锁方法及封锁协议。

12.1 并发控制概述

在第11章中已经讲到,事务是并发控制的基本单位,保证事务的ACID特性是事务处理的重要任务,而事务的ACID特性可能遭到破坏的原因之一是多个事务对数据库的并发操作互相干扰。为了保证事务的隔离性和一致性,数据库管理系统需要对并发操作进行正确调度。这也是数据库管理系统中并发控制机制的责任。

下面先通过一个例子来说明并发操作带来的数据不一致性问题。

[例 12.1] 考虑飞机订票系统中的一个活动序列:

- ① 甲售票点(事务 T_1) 读出某航班的机票余额 A , 设 $A=16$ 。
- ② 乙售票点(事务 T_2) 读出同一航班的机票余额 A , 也为 16。
- ③ 甲售票点卖出一张机票, 修改余额 $A \leftarrow A-1$, 所以 A 为 15, 把 A 写回数据库。
- ④ 乙售票点也卖出一张机票, 修改余额 $A \leftarrow A-1$, 所以 A 为 15, 把 A 写回数据库。

结果显而易见,原本卖出两张机票,而数据库中的机票余额只减少了1。

这种情况就是数据库的不一致性。这种不一致性是由并发操作引起的。在并发操作情况

下, 对 T_1 、 T_2 两个事务的操作序列的调度是随机的。若按上面的调度序列执行, 则 T_1 事务的修改被丢失, 这是由于步骤④中 T_2 事务修改 A 并写回后覆盖了 T_1 事务的修改。

下面把事务读数据 x 记为 $R(x)$, 写数据 x 记为 $W(x)$ 。

并发操作带来的数据不一致性主要有丢失修改、脏读、不可重复读、幻读等多种情况。

1. 丢失修改

丢失修改是指两个事务 T_1 和 T_2 读入同一数据, 各自进行修改, T_2 提交的结果破坏了 T_1 提交的结果, 导致 T_1 的修改被丢失, 如图 12.2(a) 所示。例 12.1 的飞机订票例子就属于此类情况。

2. 脏读

脏读 (dirty read), 俗称读“脏”数据, 是指事务 T_1 修改某一数据并将其写回磁盘, 事务 T_2 读取同一数据后, T_1 由于某种原因被撤销, 这时被 T_1 修改过的数据恢复原值, T_2 读到的数据就与数据库中的数据不一致, 则 T_2 读到的数据就为“脏”数据, 即不正确的数据。例如在图 12.2(b) 中 T_1 将 C 值修改为 200, T_2 读到 C 为 200, 而 T_1 由于某种原因被撤销, 其修改作废, C 恢复为原值 100, 这时 T_2 读到的 C 为 200, 与数据库内容不一致, 就是“脏”数据。

3. 不可重复读

不可重复读是指事务 T_1 读取数据后, 事务 T_2 执行更新操作, 当事务 T_1 再次读该数据时, 得到与前一次不同的值。例如在图 12.2(c) 中, T_1 读取 $B=100$ 进行运算, T_2 读取同一数据 B , 对其进行修改后将 $B=200$ 写回数据库。 T_1 为了对读取值校对重读 B , B 已为 200, 与第一次读取值不一致。

T_1	T_2
① $R(A)=16$	
②	$R(A)=16$
③ $A \leftarrow A-1$ $W(A)=15$	
④	$A \leftarrow A-1$ $W(A)=15$
(a) 丢失修改	
T_1	T_2
① $R(C)=100$ $C \leftarrow C+2$ $W(C)=200$	
②	$R(C)=200$
③ ROLLBACK C 恢复为 100	
(b) 脏读	
T_1	T_2
① $R(A)=50$ $R(B)=100$ 求和=150	
②	$R(B)=100$ $B \leftarrow B*2$ $W(B)=200$
③ $R(A)=50$ $R(B)=200$ 求和=250 (验算不对)	
(c) 不可重复读	

图 12.2 几种数据不一致性示例

* 4. 幻读

幻读也称作幻影 (phantom) 现象, 是指事务 T_1 读取数据后, 事务 T_2 执行插入或删除操作, 使 T_1 无法再现前一次读取结果。

幻读包括两种情况:

① 事务 T_1 按一定条件从数据库中读取某些数据记录后, 事务 T_2 删除了其中部分记录, 当

T_1 再次按相同条件读取数据时,发现某些记录“神秘地”消失了。

② 事务 T_1 按一定条件从数据库中读取某些数据记录后,事务 T_2 插入了一些记录,当 T_1 再次按相同条件读取数据时,发现多了一些记录。

本章重点讨论前三类数据不一致性。读者如果对幻读有兴趣,可以参考相关文献。

产生上述数据不一致性的主要原因是并发操作破坏了事务的隔离性。并发控制机制就是要用正确的方式调度并发操作,使一个用户事务的执行不受其他事务的干扰,从而避免造成数据的不一致性。

另一方面,对数据库的应用有时允许某些不一致性,例如有些统计工作涉及的数据量很大,读到一些“脏”数据对统计精度没什么影响,这时可以降低对一致性的要求以减少系统开销。

并发控制的主要技术有封锁(locking)、时间戳(timestamp)、乐观方法(optimistic scheduler)和多版本并发控制(multi-version concurrency control, MVCC)等。本章讲解基本的封锁技术,它也是众多数据库产品采用的基本方法。

12.2 事务的隔离级别

为防止数据出现不一致性,需要数据库管理系统对并发操作进行控制。这种控制越严格,事务的隔离性就越强,数据的一致性就越有保障,但系统的效率也会随之下降。在实际应用中,不同应用场景对数据一致性的要求并不相同,所以在 SQL 标准中给出了事务的 4 类隔离级别,以满足不同应用场景的需求。这 4 类隔离级别由低到高分别是:读未提交、读已提交、可重复读和可串行化。它们都能有效避免丢失修改,但对其他数据一致性的保障程度各异。

“读未提交”是允许一个事务可以读取另一个未提交事务正在修改的数据。它可能出现脏读、不可重复读和幻读的情形。

“读已提交”是只允许一个事务读其他事务已提交的数据。显然,它可以有效避免脏读,但是它不能保证可重复读和不幻读。

“可重复读”是一个事务开始读取数据后,其他事务就不能再对该数据执行 UPDATE 操作了。由于脏读和不可重复读是因一个事务读取数据,而另一个事务对该数据进行修改操作造成的,“可重复读”杜绝了这种情形的产生。但此时其他事务仍然可以执行 INSERT 和 DELETE 操作,所以它不能保证不幻读。

“可串行化”是最高的事务隔离级别。在该级别下,事务执行顺序是可串行化的,可以避免丢失修改、脏读、不可重复读和幻读。事务的可串行化将在 12.6 节中详细介绍。

事务的 4 个隔离级别与数据不一致性的关系见表 12.1 所示。

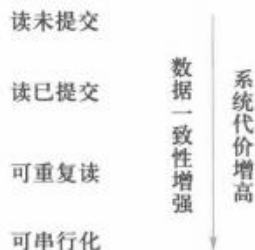
例如,若数据库管理系统的事务隔离级别是“读未提交”,则不可能产生丢失修改,但可能产生脏读、不可重复读和幻读的情况。若为“读已提交”,则不会产生丢失修改、脏读,但可能产生不可重复读和幻读的情况。目前大多数数据库默认的事务隔离级别是“读已提交”,如 Kingbase、SQL Server、Oracle 等;而 MySQL 的默认级别是“可重复读”。

表 12.1 事务隔离级别与数据不一致性的关系

事务隔离级别	数据不一致性			
	丢失修改	脏读	不可重复读	* 幻读
读未提交	否	是	是	是
读已提交	否	否	是	是
可重复读	否	否	否	是
可串行化	否	否	否	否

事务隔离级别并不是越高越好，它与数据一致性以及系统代价的关系如图 12.3 所示。数据库管理系统通常都提供了设置事务隔离级别的方法，在实际中，用户应根据应用的特点和需求选择合适的事务隔离级别。

二维码所示内容通过实验来演示 4 种事务隔离级别下丢失修改、脏读、不可重复读等数据不一致的情况，读者可以直观地感受各种隔离级别在事务中的作用。



扩展阅读：隔离级别实验

图 12.3 事务隔离级别与数据一致性及系统代价的关系

12.3 封锁

封锁是实现并发控制的一种非常重要的技术。所谓封锁，就是事务 T 在对某个数据对象操作之前，先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一定的控制，在事务 T 释放它的锁之前，其他事务不能更新或读取此数据对象。例如，在例 12.1 中，事务 T_1 要修改 A，若在读出 A 前先锁住 A，其他事务就不能再读取和修改 A 了，直到 T_1 修改并写回 A 后解除了对 A 的封锁为止。这样，就不会丢失 T_1 的修改。

确切的控制由封锁的类型决定。基本的封锁类型有两种：排他型锁(exclusive lock，简称 X 锁)和共享型锁(shared lock，简称 S 锁)。

排他型锁又称为写锁(write lock)。若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A，其他任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁为止。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A。

共享型锁又称为**读锁**(read lock)。若事务 T 对数据对象 A 加上 S 锁, 则事务 T 可以读 A 但不能修改 A , 其他事务只能再对 A 加 S 锁, 而不能加 X 锁, 直到 T 释放 A 上的 S 锁为止。这就保证了其他事务可以读 A , 但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

排他型锁与共享型锁的控制方式可以用图 12.4 所示的相容矩阵 (compatibility matrix) 来表示。

T_1	T_2		
	X	S	—
X	N	N	Y
S	N	Y	Y
—	Y	Y	Y

注: Y=Yes, 相容的请求; N=No, 不相容的请求。

图 12.4 封锁类型的相容矩阵

在图 12.4 所示的封锁类型的相容矩阵中, 最左边一列表示事务 T_1 已经获得的数据对象上锁的类型, 其中横线表示没有加锁。最上面一行表示另一事务 T_2 对同一数据对象发出的封锁请求。 T_2 的封锁请求能否被满足用矩阵中的 Y 和 N 表示, 其中 Y 表示事务 T_2 的封锁要求与 T_1 已持有的锁相容, 封锁请求可以满足; N 表示 T_2 的封锁请求与 T_1 已持有的锁冲突, T_2 的请求被拒绝。

12.4 封锁协议

在运用 X 锁和 S 锁这两种基本封锁类型对数据对象加锁时, 还需要约定一些规则。例如, 何时申请 X 锁或 S 锁、持锁时间、何时释放等。这些规则称为**封锁协议**(locking protocol)。对封锁方式制定不同的规则, 就形成了各种不同的封锁协议。本节介绍三级封锁协议。对并发操作的不正确调度可能会带来丢失修改、不可重复读和脏读等数据不一致性问题, 三级封锁协议分别在不同程度上解决了这些问题, 为并发操作的正确调度提供了一定的保证。不同级别的封锁协议达到的数据一致性级别是不同的。

1. 一级封锁协议

一级封锁协议是指事务 T 在修改数据 R 之前必须先对其加 X 锁, 直到事务结束才释放。事务结束包括正常结束 (COMMIT) 和非正常结束 (ROLLBACK)。

一级封锁协议可防止丢失修改, 并保证事务 T 是可恢复的。例如图 12.5(a) 使用一级封锁协议解决了图 12.2(a) 中的丢失修改问题。

图 12.5(a) 中事务 T_1 在读 A 进行修改之前先对 A 加 X 锁, 当 T_2 再请求对 A 加 X 锁时被拒绝, T_2 只能等待 T_1 释放 A 上的锁后获得对 A 的 X 锁, 这时它读到的 A 已经是 T_1 更新过的值 15, 再按此新的 A 值进行运算, 并将结果值 $A=14$ 写回到磁盘。这样就避免了丢失 T_1 的修改。

在一级封锁协议中, 如果仅读数据而不对其进行修改是不需要加锁的, 所以它不能保证可重复读和不脏读。

2. 二级封锁协议

二级封锁协议是指在一级封锁协议基础上, 增加事务 T 在读取数据 R 之前必须先对其加 S 锁, 读完后即可释放 S 锁。

二级封锁协议除防止了丢失修改, 还可进一步防止脏读。例如图 12.5(b) 使用二级封锁协议解决了图 12.2(b) 中的脏读问题。

图 12.5(b) 中, 事务 T_1 在对 C 进行修改之前, 先对 C 加 X 锁, 修改其值后写回磁盘。这时 T_2 请求在 C 上加 S 锁, 因 T_1 已在 C 上加了 X 锁, T_2 只能等待。 T_1 因某种原因被撤销, C 恢复为原值 100, T_1 释放 C 上的 X 锁后 T_2 获得 C 上的 S 锁, 读 $C=100$ 。这就避免了 T_2 脏读。

T_1	T_2	T_1	T_2	T_1	T_2
① XLOCK A		② XLOCK C		① SLOCK A	
② R(A)=16		R(C)=100		SLOCK B	
③	XLOCK A	C=C*2		R(A)=50	
④ A=A-1	等待	W(C)=200		R(B)=100	
W(A)=15	等待	②	SLOCK C	A+B=150	
COMMIT	等待	③ ROLLBACK	等待	②	XLOCK B
UNLOCK A	等待	(C恢复为100)	等待	等待	等待
⑤	获得XLOCK A	UNLOCK C	等待	③ R(A)=50	等待
	R(A)=15	④	获得SLOCK C	R(B)=100	等待
	A=A-1	⑤	R(C)=100	A+B=150	等待
⑥	W(A)=14		COMMIT	COMMIT	等待
	COMMIT		UNLOCK C	UNLOCK A	等待
	UNLOCK A			UNLOCK B	等待
				④	获得XLOCK B
					R(B)=100
				⑤	B=B*2
					W(B)=200
					COMMIT
					UNLOCK B

(a) 没有丢失修改

(b) 不脏读

(c) 可重复读

图 12.5 使用封锁机制解决三种数据不一致性的示例

在二级封锁协议中, 由于读完数据后即可释放 S 锁, 所以它不能保证可重复读。

3. 三级封锁协议

三级封锁协议是指在一级封锁协议的基础上, 增加事务 T 在读取数据 R 之前必须先对其加 S 锁, 直到事务结束才释放。

三级封锁协议除了防止丢失修改和脏读外, 还进一步防止了不可重复读。例如图 12.5(c) 使用三级封锁协议解决了图 12.2(c) 中的不可重复读问题。

图 12.5(c) 中, 事务 T_1 在读 A 、 B 之前, 先对 A 、 B 加 S 锁, 这样其他事务只能再对 A 、 B 加 S 锁, 而不能加 X 锁, 即其他事务只能读 A 、 B , 而不能修改它们。所以当 T_2 为修改 B 而申请对

B 的 X 锁时被拒绝, 只能等待 T_1 释放 B 上的锁。 T_1 为验算再读 A 、 B , 这时读出的 B 仍是 100, 求和结果仍为 150, 即可重复读。 T_1 结束后释放 A 、 B 上的 S 锁, 此时 T_2 才能获得对 B 的 X 锁。

上述三级封锁协议的主要区别在于什么操作需要申请封锁, 以及何时释放锁(即持锁时间)。三级封锁协议可以总结为表 12.2。表中还指出了不同的封锁协议使事务达到的一致性级别是不同的, 封锁协议级别越高, 一致性程度越高。

表 12.2 不同级别的封锁协议和一致性保证

封锁协议	X 锁		S 锁		一致性保证			隔离性 级别保证
	操作结束 释放	事务结束 释放	操作结束 释放	事务结束 释放	不丢失 修改	不脏读	可重 复读	
一级封锁协议		✓			✓			读未提交
二级封锁协议		✓	✓		✓	✓		读已提交
三级封锁协议		✓		✓	✓	✓	✓	可串行化

12.5 活锁和死锁

和操作系统一样, 封锁的方法可能引起活锁(livelock)和死锁(deadlock)等问题。

12.5.1 活锁

如果事务 T_1 封锁了数据 R , 事务 T_2 又请求封锁 R , 于是 T_2 等待; T_3 也请求封锁 R , 当 T_1 释放了 R 上的封锁之后系统首先批准了 T_3 的请求, T_2 仍然等待; 然后 T_4 又请求封锁 R , 当 T_3 释放了 R 上的封锁之后系统又批准了 T_4 的请求…… T_2 有可能永远等待, 这就是活锁的情形, 如图 12.6(a) 所示。

避免活锁的简单方法是采用先来先服务的策略。当多个事务请求封锁同一数据对象时, 封锁子系统按请求封锁的先后次序对事务排队, 数据对象上的锁一旦释放就批准申请队列中第一个事务获得锁。

12.5.2 死锁

如果事务 T_1 封锁了数据 R_1 , T_2 封锁了数据 R_2 , 然后 T_1 又请求封锁 R_2 , 因 T_2 已封锁了 R_2 , 于是 T_1 等待 T_2 释放 R_2 上的锁; 接着 T_2 又申请封锁 R_1 , 因 T_1 已封锁了 R_1 , T_2 也只能等待 T_1 释放 R_1 上的锁。这样就出现了 T_1 在等待 T_2 , 而 T_2 又在等待 T_1 的局面, T_1 和 T_2 两个事务永远不能结束, 形成死锁。如图 12.6(b) 所示。

死锁的问题在操作系统和一般并行处理中已做了深入研究。目前在数据库中解决死锁问题主要有两类方法: 一类方法是采取一定措施来预防死锁的发生; 另一类方法是允许发生死锁, 采用一定手段诊断系统中有无死锁, 若有则进行解除。

T ₁	T ₂	T ₃	T ₄	T ₁	T ₂
LOCK R	⋮	⋮	⋮	LOCK R ₁	⋮
⋮	LOCK R	⋮	⋮	⋮	LOCK R ₂
⋮	等待	LOCK R	LOCK R	LOCK R ₂	⋮
UNLOCK R	等待	⋮	等待	等待	⋮
⋮	等待	LOCK R	等待	等待	LOCK R ₁
⋮	等待	⋮	等待	等待	等待
⋮	等待	UNLOCK	LOCK R	等待	等待
⋮	等待	⋮	⋮	等待	⋮

(a) 活锁

(b) 死锁

图 12.6 死锁与活锁示例

1. 死锁的预防

在数据库中，产生死锁的原因是两个或多个事务都已封锁了一些数据对象，然后又都请求对已被其他事务封锁的数据对象加锁，从而出现死等待。防止死锁的发生，其实就是要破坏产生死锁的条件。预防死锁通常有以下两种方法。

(1) 一次封锁法

一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行。例如图 12.6(b) 的例子中，如果事务 T₁ 将数据对象 R₁ 和 R₂ 一次加锁，T₁ 就可以执行下去，而 T₂ 等待。T₁ 执行完后释放 R₁、R₂ 上的锁，T₂ 继续执行。这样就不会发生死锁。

一次封锁法虽然可以有效地防止死锁的发生，但也存在如下问题：

① 一次就将以后要用到的全部数据加锁，势必扩大了封锁的范围，从而降低了系统的并发度。

② 数据库中的数据是不断变化的，原来不要求封锁的数据在执行过程中可能会变成封锁对象，所以很难事先精确地确定每个事务所要封锁的数据对象。为此只能扩大封锁范围，将事务在执行过程中可能要封锁的数据对象全部加锁，这就进一步降低了并发度。

(2) 顺序封锁法

顺序封锁法是预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实施封锁。例如在 B 树结构的索引中，可规定封锁的顺序必须是从根结点开始，然后是下一级的子结点，逐级封锁。

顺序封锁法可以有效地防止死锁，但也同样存在问题：

① 数据库系统中封锁的数据对象极多，并且随数据的插入、删除等操作而不断地变化，要维护这样的资源的封锁顺序非常困难，成本很高。

② 事务的封锁请求可以随着事务的执行而动态地决定, 很难事先确定每一个事务要封锁哪些对象, 因此也就很难按规定的顺序去施加封锁。

可见, 在操作系统中广为采用的预防死锁的策略并不太适合数据库的特点, 因此数据库管理系统在解决死锁的问题上普遍采用的是诊断并解除死锁的方法。

2. 死锁的诊断与解除

数据库系统中诊断死锁的方法与操作系统类似, 一般使用超时法或事务等待图法。

(1) 超时法

如果一个事务的等待时间超过了规定的时限, 就认为发生了死锁。超时法实现简单, 但其不足也很明显, 一是有可能误判死锁, 如事务因为其他原因而使等待时间超过时限, 系统会误认为发生了死锁; 二是时限若设置得太长, 死锁发生后不能及时发现。

(2) 事务等待图法

事务等待图是一个有向图 $G=(T,U)$ 。其中 T 为结点的集合, 每个结点表示正在运行的事务; U 为边的集合, 每条边表示事务等待的情况。若 T_1 等待 T_2 , 则在 T_1 、 T_2 之间画一条有向边, 从 T_1 指向 T_2 , 如图 12.7 所示。

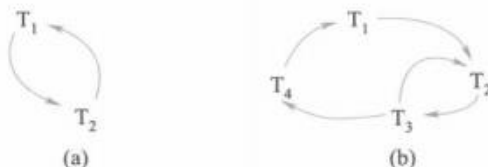


图 12.7 事务等待图示例

事务等待图动态地反映了所有事务的等待情况。并发控制子系统周期性地(比如每隔数秒)生成事务等待图并进行检测, 如果发现图中存在回路, 则表示系统中出现了死锁。

图 12.7(a)表示事务 T_1 等待 T_2 , T_2 又等待 T_1 , 产生了死锁。图 12.7(b)表示事务 T_1 等待 T_2 , T_2 等待 T_3 , T_3 等待 T_4 , T_4 又等待 T_1 , 产生了死锁。

当然, 死锁的情况可以多种多样。例如, 图 12.7(b)中事务 T_3 可能还等待 T_2 , 在大回路中又有小的回路。这些情况人们都已做了很深入的研究。

数据库管理系统的并发控制子系统一旦检测到系统中存在死锁, 就要设法解除。通常采用的方法是选择一个处理死锁代价最小的事务, 将其撤销, 释放此事务持有的所有锁, 使其他事务得以继续运行下去。当然, 对撤销的事务所执行的数据修改操作必须加以恢复。

12.6 并发调度的可串行性

数据库管理系统对并发事务不同的调度可能会产生不同的结果, 那么什么样的调度是正确的呢? 显然, 串行调度是正确的。执行结果等价于串行调度的调度也是正确的。这样的调度称为可串行化调度。

12.6.1 可串行化调度

多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同，称这种调度策略为可串行化调度。

可串行性是并发事务正确调度的准则。按这个准则规定，一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度。

[例 12.2] 现有两个事务，分别包含下列操作：

事务 T_1 ：读 B ； $A=B+1$ ；写回 A ；

事务 T_2 ：读 A ； $B=A+1$ ；写回 B 。

假设 A 、 B 的初值均为 2。按 $T_1 \rightarrow T_2$ 次序执行结果为 $A=3$ ， $B=4$ ；按 $T_2 \rightarrow T_1$ 次序执行结果为 $B=3$ ， $A=4$ 。

图 12.8 给出了对这两个事务不同的调度策略。其中，图 12.8(a) 和图 12.8(b) 为两种不同的串行调度策略，虽然执行结果不同，但它们都是正确的调度。图 12.8(c) 的执行结果与图 12.8(a)(b) 的结果都不同，所以是错误的调度。图 12.8(d) 的执行结果与图 12.8(a) 串行调度的执行结果相同，所以是正确的调度。

T_1	T_2	T_1	T_2	T_1	T_2	T_1	T_2
SLOCK B			SLOCK A	SLOCK B		SLOCK B	
$Y=R(B)=2$			$X=R(A)=2$	$Y=R(B)=2$		$Y=R(B)=2$	
UNLOCK B			UNLOCK A		SLOCK A	UNLOCK B	
XLOCK A			XLOCK B		$X=R(A)=2$	XLOCK A	
$A=Y+1=3$			$B=X+1=3$	UNLOCK B			SLOCK A
$W(A)$			$W(B)$		UNLOCK A	$A=Y+1=3$	等待
UNLOCK A			UNLOCK B	XLOCK A		$W(A)$	等待
	SLOCK A	SLOCK B		$A=Y+1=3$		UNLOCK A	等待
	$X=R(A)=3$	$Y=R(B)=3$		$W(A)$			$X=R(A)=3$
	UNLOCK A	UNLOCK B			XLOCK B		UNLOCK A
	XLOCK B	XLOCK A			$B=X+1=3$		XLOCK B
	$B=X+1=4$	$A=Y+1=4$			$W(B)$		$B=X+1=4$
	$W(B)$	$W(A)$		UNLOCK A			$W(B)$
	UNLOCK B	UNLOCK A			UNLOCK B		UNLOCK B

(a) 串行调度1

(b) 串行调度2

(c) 不可串行化的调度

(d) 可串行化的调度

图 12.8 并发事务的不同调度

12.6.2 冲突可串行化调度

具有什么性质的调度是可串行化的调度？如何判断一个调度是否可串行化？本节给出判断可串行化调度的充分条件。

首先介绍冲突操作的概念。

冲突操作是指不同的事务对同一个数据的读写操作和写写操作：

$R_i(x)$ 与 $W_j(x)$ /* 事务 T_i 读 x , T_j 写 x , 其中 $i \neq j$ */

$W_i(x)$ 与 $W_j(x)$ /* 事务 T_i 写 x , T_j 写 x , 其中 $i \neq j$ */

其他操作是不冲突操作。

不同事务的冲突操作和同一事务的两个操作是不能互换 (swap) 的。对于 $R_i(x)$ 与 $W_j(x)$, 若改变二者的次序, 则事务 T_i 看到的数据库状态就发生了改变, 自然会影响到事务 T_i 后面的行为。对于 $W_i(x)$ 与 $W_j(x)$, 改变二者的次序也会影响数据库的状态, x 的值由等于 T_j 的结果变成了等于 T_i 的结果。

一个调度 Sc 在保证冲突操作的次序不变的情况下, 通过交换两个事务不冲突操作的次序得到另一个调度 Sc' , 如果 Sc' 是串行的, 称调度 Sc 为冲突可串行化的调度。若一个调度是冲突可串行化的, 则一定是可串行化的调度。因此, 可以用这种方法来判断一个调度是否为冲突可串行化的。

[例 12.3] 今有调度 $Sc_1 = R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$, 假设 T_1 和 T_2 均正常提交。

可以把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到

$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$

再把 $R_2(A)$ 与 $R_1(B)W_1(B)$ 交换

$Sc_2 = R_1(A)W_1(A)R_1(B)W_1(B)R_2(A)W_2(A)R_2(B)W_2(B)$

Sc_2 等价于一个串行调度 T_1, T_2 。所以 Sc_1 为冲突可串行化的调度 (详细讲解参见二维码内容)。



微视频：冲突可
串行化判断示例

应该指出的是, 冲突可串行化调度是可串行化调度的充分条件, 不是必要条件。还有不满足冲突可串行化条件的可串行化调度。

[例 12.4] 有三个事务 $T_1 = W_1(Y)W_1(X)$, $T_2 = W_2(Y)W_2(X)$, $T_3 = W_3(X)$ 。

调度 $L_1 = W_1(Y)W_1(X)W_2(Y)W_2(X)W_3(X)$ 是一个串行调度。

调度 $L_2 = W_1(Y)W_2(Y)W_2(X)W_1(X)W_3(X)$ 不满足冲突可串行化。但是调度 L_2 是可串行化的, 因为 L_2 执行的结果与调度 L_1 相同, Y 的值都等于 T_2 的值, X 的值都等于 T_3 的值。

前面已经讲到, 商用数据库管理系统的并发控制一般采用封锁的方法来实现, 那么如何使封锁机制能够产生可串行化调度呢? 下面将要讲解的两段锁协议就可以实现可串行化调度。

12.7 两段锁协议

为了保证并发调度的正确性, 数据库管理系统的并发控制机制必须提供一定的手段来保证调度是可串行化的。目前数据库管理系统普遍采用两段封锁^① (two-phase lock, 2PL, 简称两段

① 《计算机科学技术名词》第3版中译为“两阶段锁”。

锁)协议的方法来实现并发调度的可串行性,从而保证调度的正确性。

所谓两段锁协议,是指所有事务必须分两个阶段对数据项加锁和解锁。

① 在对任何数据进行读、写操作之前,首先要申请并获得对该数据的封锁。

② 在释放一个封锁之后,事务不再申请和获得任何其他封锁。

上述两个阶段中,第一阶段是获得封锁,也称为扩展阶段。在这个阶段,事务可以申请获得任何数据项上的任何类型的锁,但是不能释放任何锁。第二阶段是释放封锁,也称为收缩阶段。在这个阶段,事务可以释放任何数据项上的任何类型的锁,但是不能再申请任何锁。

例如,事务 T_1 遵守两段锁协议,其封锁序列是

SLOCK A SLOCK B XLOCK C UNLOCK B UNLOCK A UNLOCK C;

| ←——— 扩展阶段 ———→ | | ←——— 收缩阶段 ———→ |

又如,事务 T_2 不遵守两段锁协议,其封锁序列是

SLOCK A UNLOCK A SLOCK B XLOCK C UNLOCK C
UNLOCK B;

可以证明,若并发执行的所有事务均遵守两段锁协议,则对这些事务的任何并发调度策略都是可串行化的。

例如,图 12.9 所示的调度是遵守两段锁协议的,因此一定是一个可串行化调度。可以验证如下:忽略图中的加锁操作和解锁操作,按时间的先后次序可得到如下的调度:

$L_1 = R_1(A)R_2(C)W_1(A)W_2(C)R_1(B)W_1(B)R_2(A)W_2(A)$

通过交换两个不冲突操作的次序(先把 $R_2(C)$ 与 $W_1(A)$ 交换,再把 $R_1(B)W_1(B)$ 与 $R_2(C)W_2(C)$ 交换),可得到

$L_2 = R_1(A)W_1(A)R_1(B)W_1(B)R_2(C)W_2(C)R_2(A)W_2(A)$

因此 L_1 是一个可串行化调度。

需要说明的是,事务遵守两段锁协议是可串行化调度的充分条件,而不是必要条件。也就是说,若并发事务都遵守两段锁协议,则对这些事务的任何并发调度策略都是可串行化的。但是,若并发事务的一个调度是可串行化的,不一定所有事务都符合两段锁协议。例如图 12.8(d)是可串行化的调度,但 T_1 和 T_2 不遵守两段锁协议。

另外,要注意两段锁协议和防止死锁的一次封锁法的异同之处。一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁,否则就不能继续执行,因此一次封锁法遵守两段锁协议。但是两段锁协议并不要求事务必须一次将所有

事务 T_1	事务 T_2
SLOCK A $R(A)=260$	SLOCK C $R(C)=300$
XLOCK A $W(A)=160$	XLOCK C $W(C)=250$
SLOCK B $R(B)=1000$	SLOCK A 等待
XLOCK B $W(B)=1100$	等待
UNLOCK A	等待
	$R(A)=160$
	XLOCK
UNLOCK B	$W(A)=210$
	UNLOCK C
	UNLOCK A

图 12.9 遵守两段锁协议的可串行化调度

要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁，如图 12.10 所示。

事务 T_1	事务 T_2
SLOCK B $R(B)=2$	
	SLOCK A $R(A)=2$
XLOCK A 等待 等待	XLOCK B 等待

图 12.10 遵守两段锁协议的事务可能发生死锁

12.8 封锁的粒度

封锁对象的大小称为**封锁粒度**(lock granularity)。封锁对象可以是逻辑单元，也可以是物理单元。以关系数据库为例，封锁对象可以是这样一些逻辑单元：属性值，属性值的集合，元组，关系，索引项，整个索引直至整个数据库；也可以是这样一些物理单元：页(数据页或索引页)，物理记录等。

封锁粒度与系统的并发度和并发控制的开销密切相关。直观地看，封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；反之，封锁的粒度越小，并发度越高，但系统开销也就越大。

例如，若封锁粒度是数据页，事务 T_1 需要修改元组 L_1 ，则 T_1 必须对包含 L_1 的整个数据页 A 加锁。如果 T_1 对 A 加锁后事务 T_2 要修改 A 中的元组 L_2 ，则 T_2 被迫等待，直到 T_1 释放 A 上的锁。如果封锁粒度是元组，则 T_1 和 T_2 可以分别对 L_1 和 L_2 加锁，不需要互相等待，从而提高了系统的并行度。又如，事务 T 需要读取整个表，若封锁粒度是元组， T 必须对表中的每一个元组加锁，显然开销极大。

因此，在一个系统中如能同时支持多种封锁粒度供不同的事务选择是比较理想的，这种封锁方法称为**多粒度封锁**。选择封锁粒度时应同时考虑封锁开销和并发度两个因素，适当选择封锁粒度以求得最优的效果。一般说来，需要处理某个关系的大量元组的事务，可以以关系为封锁粒度；需要处理多个关系的大量元组的事务，可以以数据库为封锁粒度；而对于一个处理少量元组的用户事务，则以元组为封锁粒度就比较合适。

12.8.1 多粒度封锁

首先定义**多粒度树**。多粒度树的根结点是整个数据库，表示最大的数据粒度。叶结点表示

最小的数据粒度。

图 12.11 给出了一个三级粒度树。根结点为数据库，数据库的子结点为关系，关系的子结点为元组。



图 12.11 三级粒度树

多粒度封锁协议允许多粒度树中的每个结点被独立地加锁。对一个结点加锁，意味着这个结点的所有后裔结点也被加以同样类型的锁。因此，在多粒度封锁中一个数据对象可能以两种方式封锁，显式封锁和隐式封锁。

显式封锁是指应事务的要求直接加到数据对象上的锁。**隐式封锁**是指该数据对象没有被独立加锁，是因其上级结点加锁而使该数据对象加上了锁。

多粒度封锁方法中，显式封锁和隐式封锁的效果是一样的。因此，系统检查封锁冲突时不仅要检查显式封锁，还要检查隐式封锁。例如，事务 T 要对关系 R_1 加 X 锁，系统必须搜索其上级结点数据库、关系 R_1 以及 R_1 的下级结点，即 R_1 中的每一个元组，上下搜索。如果其中某一个数据对象已经加了不相容锁，则 T 必须等待。

一般地，对某个数据对象加锁，系统先要检查该数据对象上是否有显式封锁与之冲突，再检查其所有上级结点，看本事务的显式封锁是否与该数据对象上的隐式封锁（即由于上级结点已加的封锁造成的）冲突；还要检查其所有下级结点，看它们的显式封锁是否与本事务的隐式封锁（将加到下级结点的封锁）冲突。显然，这样的检查方法效率很低。为此人们引进了一种新型锁，称为**意向锁**（intention lock）。有了意向锁，数据库管理系统就无须逐个检查下一级结点的显式封锁了。

12.8.2 意向锁

如果对一个结点加意向锁，则说明该结点的后裔结点正在被加锁；对任一结点加锁时，必须先对它的上层结点加意向锁。

例如，对任一元组加锁时，必须先对它所在的数据库和关系加意向锁。

下面介绍三种常用的意向锁：意向共享型锁（intent shared lock，简称 IS 锁）、意向排他型锁（intent exclusive lock，简称 IX 锁）和共享意向排他型锁（shared and intention exclusive lock，简称 SIX 锁）。

① **IS 锁**。如果对一个数据对象加 IS 锁，表示它的后裔结点拟（意向）加 S 锁。例如，事务

T_i 要对 R_i 中某个元组加 S 锁,则要首先对关系 R_i 和数据库加 IS 锁。

② **IX 锁**。如果对一个数据对象加 IX 锁,表示它的后裔结点拟(意向)加 X 锁。例如,事务 T_i 要对 R_i 中某个元组加 X 锁,则要首先对关系 R_i 和数据库加 IX 锁。

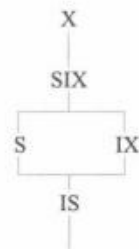
③ **SIX 锁**。如果对一个数据对象加 SIX 锁,表示对它加 S 锁,再加 IX 锁,即 $SIX = S + IX$ 。例如对某个表加 SIX 锁,则表示该事务要读整个表(所以要对该表加 S 锁),同时会更新个别元组(所以要对该表加 IX 锁)。

图 12.12(a)给出了这些锁的相容矩阵,从中可以发现这 5 种锁的强度有如图 12.12(b)所示的偏序关系。所谓锁的强度是指它对其他锁的排斥程度。一个事务在申请封锁时以强锁代替弱锁是安全的,反之则不然。

T_i	T_j					
	S	X	IS	IX	SIX	—
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
—	Y	Y	Y	Y	Y	Y

注: Y=Yes,表示相容的请求;N=No,表示不相容的请求

(a) 封锁类型的相容矩阵



(b) 锁的强度偏序关系

图 12.12 加意向锁后锁的相容矩阵与偏序关系

在具有意向锁的多粒度封锁方法中,任意事务 T 要对一个数据对象加锁,必须先对它的上层结点加意向锁。申请封锁时应按自上而下的次序进行,释放封锁时则应按自下而上的次序进行。

例如,事务 T_i 要对关系 R_i 加 S 锁,则要首先对数据库加 IS 锁。检查数据库是否已加了不相容的锁(X 锁),然后检查 R_i 是否已加了不相容的锁(X 或 IX 锁),不再需要搜索和检查 R_i 中的各元组是否加了不相容的锁(X 锁)。

具有意向锁的多粒度封锁方法提高了系统的并发度,减少了加锁和解锁的开销,已在实际的数据库管理系统产品中得到广泛应用。

*12.9 其他并发控制机制

并发控制的方法除了封锁技术外还有时间戳方法、乐观方法和多版本并发控制方法等。这里做一个概要的介绍。

时间戳方法给每一个事务“盖”上一个时标,即事务开始执行的时间。每个事务具有唯一的时间戳,并按照这个时间戳来解决事务的冲突操作。如果发生冲突操作,就回滚具有较早时

间截的事务，以保证其他事务的正常执行，被回滚的事务被赋予新的时间戳并从头开始执行。

乐观方法认为事务执行时很少发生冲突，因此不对事务进行特殊的管制，而是让它自由执行，事务提交前再进行正确性检查。如果检查后发现该事务执行中出现过冲突并影响了可串行性，则拒绝提交并回滚该事务。乐观方法又被称为验证方法。

多版本并发控制(MVCC)方法是指在数据库中通过维护数据对象的多个版本信息来实现高效并发控制的一种策略。

12.9.1 多版本并发控制方法

版本(version)是指数据库中数据对象的一个快照，记录了数据对象某个时刻的状态。随着计算机系统存储设备价格的不断降低，可以考虑为数据库系统的数据对象保留多个版本，以提高系统的并发操作程度。例如，有一个数据对象 $A=5$ ，有两个事务 T_1 、 T_2 ，其中 T_1 是写事务， T_2 是读事务。假定先启动 T_1 ，后启动 T_2 。按照传统的封锁协议， T_2 必须等待 T_1 执行结束释放 A 上的封锁后才能获得对 A 的封锁。也就是说， T_1 和 T_2 实际上是串行执行的，如图 12.13(a) 所示。如果在 T_1 准备写 A 时不是等待，而是为 A 生成一个新的版本(表示为 A')，那么 T_2 就可以继续在 A' 上执行。只是在 T_2 准备提交时要检查 T_1 是否已经完成。如果 T_1 已经完成了， T_2 就可以放心地提交；如果 T_1 还没有完成，那么 T_2 必须等待直到 T_1 完成。这样既能保持事务执行的可串行性，又提高了事务执行的并行度。这就是多版本并发控制方法的原理，如图 12.13(b) 所示。



图 12.13 封锁方法与多版本并发控制方法示意图

在多版本并发控制方法中，每个 $\text{write}(Q)$ 操作都创建 Q 的一个新版本，这样一个数据对象就有一个版本序列 $Q_1, Q_2, \dots, Q_m$ 与之相关联。每一个版本 Q_k 拥有版本的值、创建 Q_k 的事务的时间戳 $W\text{-timestamp}(Q_k)$ 和成功读取 Q_k 的事务的最大时间戳 $R\text{-timestamp}(Q_k)$ 。其中， $W\text{-tim-}$

$estamp(Q)$ 表示在数据项 Q 上成功执行 $write(Q)$ 操作的所有事务中的最大时间戳, $R-timestamp(Q)$ 表示在数据项 Q 上成功执行 $read(Q)$ 操作的所有事务中的最大时间戳。

用 $TS(T)$ 表示事务 T 的时间戳, $TS(T_i) < TS(T_j)$ 表示事务 T_i 在事务 T_j 之前开始执行。MVCC 协议描述如下:

假设版本 Q_k 具有小于或等于 $TS(T)$ 的最大时间戳。

若事务 T 发出 $read(Q)$, 则返回版本 Q_k 的内容。

若事务 T 发出 $write(Q)$, 则:

当 $TS(T) < R-timestamp(Q_k)$ 时, 回滚 T ;

当 $TS(T) = W-timestamp(Q_k)$ 时, 覆盖 Q_k 的内容。

否则, 创建 Q 的新版本。

若一个数据对象的两个版本 Q_k 和 Q_l , 其 $W-timestamp$ 都小于系统中时间最长的事务的时间戳, 那么这两个版本中较旧的那个版本将不再被用到, 因而可以从系统中删除。

多版本并发控制方法利用物理存储上的多版本来维护数据的一致性。这就意味着当检索数据库时, 每个事务都能看到一个数据的一段时间前的快照, 而不管正在处理的数据当前的状态。多版本并发控制方法和封锁方法相比, 主要的好处是消除了数据库中数据对象读和写操作的冲突, 有效地提高了系统的性能。

多版本并发控制方法有利于提高事务的并发度, 但也会产生大量的无效版本; 而且在事务结束时刻, 其所影响的元组的有效性不能马上确定, 这就为保存事务执行过程中的状态提出了难题。这些都是实现多版本并发控制方法的一些关键技术。

12.9.2 改进的多版本并发控制方法

多版本并发控制方法可以进一步改进。区分事务的类型为只读事务和更新事务。对于只读事务, 发生冲突的可能性很小, 可以采用多版本时间戳。对于更新事务, 采用较保守的两段锁(2PL)协议。这样的混合协议称为 MV2PL。具体做法如下。

除了传统的共享型锁(S锁)和排他型锁(X锁)外, 引进一个新的封锁类型, 称为验证锁(certify-lock, 简称 C 锁)。验证锁的相容矩阵如图 12.14 所示。

T_1	T_2		
	S	X	C
S	Y	Y	N
X	Y	N	N
C	N	N	N

注: Y=Yes, 表示相容的请求; N=No, 表示不相容的请求

图 12.14 验证锁的相容矩阵

注意：在这个相容矩阵中，S 锁和 X 锁变得是相容的了。这样当某个事务写数据对象时，允许其他事务读数据（当然，写操作将生成一个新的版本，而读操作就是在旧版本上读）。一旦写事务要提交，必须首先获得在那些加了 X 锁的数据对象上的 C 锁。由于 C 锁和 S 锁是不相容的，所以为了得到 C 锁，写事务不得不延迟提交，直到所有被它加上 X 锁的数据对象都被所有那些正在读它们的事务释放。一旦写事务获得 C 锁，系统就可以丢弃数据对象的旧值，代之以新版本，然后释放 C 锁，提交事务。

在这里，系统最多只要维护数据对象的两个版本，多个读操作可以和一个写操作并发地执行。这种情况是传统的两段锁所不允许的，从而提高了读写事务之间的并发度。

目前很多商用数据库管理系统，如 Oracle、Kingbase ES 都采用的是 MV2PL 协议。

MV2PL 协议把封锁机制和时间戳方法相结合，维护一个数据的多个版本，即对于关系表上的每一个写操作产生一个新版本，同时会保存前一次修改的数据版本。MV2PL 协议和封锁机制相比，主要的好处是在多版本并发控制中对读数据的锁要求与写数据的锁要求不冲突，所以读不会阻塞写，而写也从不阻塞读，从而使读写操作没有冲突，有效地提高了系统并发性。

现在许多数据库产品都使用了多版本并发控制技术，但是这些产品的实现细节各不相同，有兴趣的读者可参考相关文献。

本章小结

数据库的重要特征是能为多个用户提供数据共享。数据库管理系统必须提供并发控制机制来协调并发用户的并发操作，以保证并发事务的隔离性和一致性，确保数据库的一致性。

数据库的并发控制以事务为单位，通常使用封锁技术实现并发控制。本章介绍了最常用的封锁方法和三级封锁协议。不同的封锁和不同级别的封锁协议所提供的系统一致性保证是不同的。对数据对象施加封锁会带来活锁和死锁问题，数据库一般采用先来先服务、死锁诊断和解除等技术来预防活锁和死锁的发生。并发控制机制调度并发事务操作是否正确的判别准则是可串行性，两段锁是可串行化调度的充分条件，但不是必要条件。因此，两段锁可以保证并发事务调度的正确性。

不同的数据库管理系统提供的封锁类型、封锁协议、达到的数据一致性级别不尽相同，但是其依据的基本原理和技术是共同的。作为选读内容，本章还简要介绍了时间戳方法、乐观方法和多版本并发控制方法等其他并发控制方法。

习 题 12

1. 在数据库中为什么要采用并发控制？并发控制技术能保证事务的哪些特性？
2. 并发操作可能会产生哪几类数据不一致？用什么方法能避免各种不一致的情况？
3. 事务的隔离级别都有哪些，事务隔离级别与数据一致性的关系是什么？

4. 什么是封锁？基本的封锁类型有几种？试述它们的含义。
5. 如何用封锁机制保证数据的一致性？
6. 什么是活锁？试述活锁的产生原因和解决方法。
7. 什么是死锁？请给出预防死锁的若干方法。
8. 请给出检测死锁发生的一种方法。当发生死锁后如何解除死锁？
9. 什么样的并发调度是正确的调度？
10. 设 T_1 、 T_2 、 T_3 是如下的三个事务 (A 的初值为 0)。
 $T_1: A := A + 2;$
 $T_2: A := A * 2;$
 $T_3: A := A ** 2; \text{ (即 } A \leftarrow A^2 \text{)}$
 - ① 若这三个事务允许并发执行，则有多少种可能的正确结果？请一一列举出来。
 - ② 请给出一个可串行化的调度，并给出执行结果。
 - ③ 请给出一个非串行化的调度，并给出执行结果。
 - ④ 若这三个事务都遵守两段锁协议，请给出一个不产生死锁的可串行化调度。
 - ⑤ 若这三个事务都遵守两段锁协议，请给出一个产生死锁的调度。
11. 今有三个事务的一个调度 $R_3(B)R_1(A)W_3(B)R_2(B)R_2(A)W_2(B)R_1(B)W_1(A)$ ，该调度是冲突可串行化的调度吗？为什么？
 12. 试证明若并发事务遵守两段锁协议，则对这些事务的并发调度是可串行化的。
 13. 举例说明对并发事务的一个调度是可串行化的，而这些并发事务不一定遵守两段锁协议。
 14. 考虑如下的调度，说明这些调度集合之间的包含关系。
 - ① 正确的调度。
 - ② 可串行化的调度。
 - ③ 遵循两段锁的调度。
 - ④ 串行调度。
15. 考虑如下的 T_1 和 T_2 两个事务。
 $T_1: R(A); R(B); B = A + B; W(B)$
 $T_2: R(B); R(A); A = A + B; W(A)$
 - ① 改写 T_1 和 T_2 ，增加加锁操作和解锁操作，并要求遵循两段锁协议。
 - ② 说明 T_1 和 T_2 的执行是否会引起死锁，给出 T_1 和 T_2 的一个调度并说明之。
16. 为什么要引进意向锁？意向锁的含义是什么？
17. 试述常用的意向锁 (IS 锁、IX 锁、SIX 锁)，给出这些锁的相容矩阵。

第12章实验 并发控制

并发控制实验。掌握数据库并发控制的基本原理及应用方法。验证并发操作带来的数据不一致性问题，包括丢失修改、不可重复读和脏读等情况。要求通过取消查询分析器的自动提交功能，创建两个不同的用户，分别登录查询分析器，同时打开两个客户端；通过使用 SQL 语句设计具体例子，展示不同封锁级别的应用场景，验证各种封锁级别的并发控制效果，以进一步理解封锁技术如何解决事务并发导致的问题。

参考文献 12

- [1] BERNSTEIN P A, HADZILACOS V, GOODMAN N. Concurrency control and recovery in data base systems [M]. Addison Wesley, 1988.
- [2] ESWARAN K P, GRAY J N, LORIE R A, et al. The notions of consistency and predicate locks in a data base system[J]. Communications of the ACM, 1976, 19(11): 624-633.
- 文献[2]给出了可串行性概念的形式化定义。
- [3] GRAY J N, LORIE R A, PUTZOLU G R. Granularity of locks in a large shared data base[C]. Proceedings of the 1st International Conference on Very Large Data Bases, 1975: 428-451.
- 文献[3]综述了多粒度封锁协议。
- [4] GRAY J, REUTER A. Transaction processing: concepts and techniques[M]. San Francisco: Morgan Kaufmann, 1992.
- [5] PAPADIMITRIOU C H. The Serializability of concurrent database updates[J]. Journal of the ACM, 1979, 26(4): 631-653.
- 文献[5]给出了与可串行性相关的研究结果。
- [6] KEDEM Z M, SILBERSCHATZ A. Locking protocols: from exclusive to shared locks[J]. Journal of the ACM, 1983, 30(4): 787-804.
- 文献[6]讨论了封锁协议。
- [7] BAYER R, SCHKOLNICK M. Concurrency of operating on B-trees[J]. Acta Informatica, 1977, 9: 1.
- 文献[7]提出了对B树进行并发存取的算法。
- [8] LEHMAN P L, YAO S B. Efficient locking for concurrent operations on B-trees[J]. ACM Transactions on Database Systems, 1981, 6(4): 650-670.
- [9] BERNSTEIN P A, GOODMAN N. Timestamp-based algorithms for concurrency control in distributed database systems[C]. Proceedings of the 6th International Conference on Very Large Data Bases, 1980, 6: 285-300.
- 文献[9]讨论了各种基于时间戳的并发控制算法。
- [10] BERNSTEIN P A, GOODMAN N. Timestamp-based algorithms for concurrency control in distributed database systems[C]. Proceedings of the 6th International Conference on Very Large Databases, 1980: 285-300.
- [11] KUNG H T, ROBINSON J T. On optimistic methods for concurrency control[J]. ACM Transaction on Database Systems, 1981, 6(2): 213-226.
- [12] SILBERSCHATZ A. A multi-version concurrency scheme with no rollbacks[C]. Proceedings of the 1st ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, 1982: 216-223.
- [13] BERNSTEIN P A, GOODMAN N. Multi-version concurrency Control-Theory and algorithms[J]. ACM Transaction on Database Systems, 1983, 8(4): 465-483.
- [14] CAREY M J. Multiple versions and the performance of optimistic concurrency control[R]. Computer Sciences Technical Report 517, 1983.
- [15] FALEIRO J M, ABADI D J. Rethinking serializable multiversion concurrency control[C]. Proceedings of the VLDB Endowment, 2015, 8(11): 1190-1201.
- 文献[15]提出了一种新的多版本数据库并发控制协议。

本章知识点讲解微视频：



数据异常与隔离
级别



封锁与封锁协议



可串行化与冲突
可串行化



两段锁协议

* 第 13 章 数据库管理系统概述

本章导读

数据库管理系统是对数据库中的共享数据进行有效的组织、存储、管理和存取的软件系统。本章主要阐述数据库管理系统的基本功能、系统结构及主要实现技术。

本章首先介绍数据库管理系统的基本功能，然后讲解数据库管理系统的语言处理层、数据存取层和数据存储层中缓冲区的层次结构，以及这三层结构是如何配合完成数据库查询处理请求的。最后介绍数据库的物理组织。

本章不是针对数据库管理系统的设计人员编写的，而是面向数据库管理员和数据库应用系统开发人员的，目的是使他们从宏观和总体的角度掌握数据库管理系统的基本概念和基本原理，以便更好地使用和维护数据库管理系统。

13.1 数据库管理系统的基本功能

数据库管理系统已经发展成为继操作系统之后最复杂的系统软件。前面已讲过，数据库管理系统主要是实现对共享数据有效的组织、存储、管理和存取。围绕数据，数据库管理系统应具有如下基本功能。

① **数据库定义和创建。**创建数据库主要是用数据定义语言定义和创建数据库模式、外模式、内模式等数据库对象。在关系数据库中就是建立数据库(或模式)、表、视图、索引等，还有创建用户、定义安全保密(如用户口令、级别、角色、存取权限)、定义数据库的完整性约束。这些定义存储在数据字典(亦称为系统目录)中，是数据库管理系统运行的基本依据。

② **数据组织、存储和管理。**数据库管理系统要分类组织、存储和管理各种数据，包括数据字典、用户数据、存取路径等；要确定以何种文件结构和存取方式在存储器上组织这些数据，以及如何实现数据之间的联系。数据组织和存储的基本目标是提高存储空间利用率和方便存取，提供多种存取方法(如索引查找、哈希查找、顺序查找等)以提高存取效率。

③ **数据存取。**数据库管理系统提供用户对数据的操作功能，实现对数据库数据的检索、